МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ)

Институт №8 «Компьютерные науки и прикладная математика»
Кафедра 806 «Вычислительная математика и программирование»

Лабораторная работа №3
по курсу «Операционные системы»

Решение системы линейных уравнений методом Гаусса в многопоточном режиме

Вариант 10

Выполнил: _____

Группа: _____

Преподаватель: _____

Москва, 2026

Условие

Цель работы: получить практические навыки управления потоками операционной системы и синхронизации между ними.

Задание: разработать программу на языке C, обрабатывающую данные в многопоточном режиме с помощью стандартных средств ОС. Максимальное число одновременно работающих потоков задаётся ключом запуска. Необходимо показать число потоков средствами ОС, исследовать зависимость ускорения и эффективности от размера данных и числа потоков и объяснить результаты.

Вариант 10: решить систему линейных уравнений методом Гаусса. Исходным заданием служит предоставленный PDF «Лабораторная работа 3». Дополнительно реализованы чтение системы из файла, воспроизводимая генерация, проверка решения и демонстрационная пауза. Оформление адаптировано из шаблона Yadroff/OS_report_template. Отдельный Word-файл с требованиями к этой работе не был приложен.

Метод решения

Требуется найти вектор x из равенства $Ax = b$, где A — квадратная матрица размера n, а b — вектор правых частей. В памяти хранится расширенная матрица $[A|b]$ размером $n \times (n+1)$, тип элементов — double. Индексы в формулах ниже, как и в C, начинаются с нуля.

На шаге k прямого хода выбирается строка с наибольшим $|a_{ik}|$ среди $i = k, \ldots, n-1$. Она переставляется с ведущей строкой k. Такой частичный выбор главного элемента уменьшает опасность деления на очень малое число. Для каждой строки $i > k$ вычисляется

$$m_i = \frac{a_{ik}}{a_{kk}}, \qquad a_{ij} \leftarrow a_{ij} - m_i a_{kj}, \quad j = k+1, \ldots, n.$$

Элемент a_{ik} становится нулём. Столбец $j = n$ — свободные члены; он изменяется вместе с коэффициентами. Эквивалентные преобразования строк сохраняют решения системы. После прямого хода матрица коэффициентов становится верхнетреугольной.

Обратный ход выполняется от последней строки к первой:

$$x_i = \frac{a_{i,n} - \sum_{j=i+1}^{n-1} a_{ij} x_j}{a_{ii}}.$$

Последовательная версия использует тот же код исключения строк при одном участнике. Основная трудоёмкость прямого хода — $O(n^3)$, обратного — $O(n^2)$, память — $O(n^2)$. Сохраняется копия исходной матрицы для проверки.

Порог ведущего элемента равен $\varepsilon n \max_{i,j<n} |a_{ij}|$, где $\varepsilon = \mathrm{DBL_EPSILON}$. Если модуль опоры не превосходит порог или не является конечным числом, решение прекращается. Так обнаруживаются вырожденность и часть численно опасных случаев. Программа не различает несовместную систему и систему с бесконечным числом решений: в обоих случаях безопасное единственное решение недоступно. Это консервативный критерий, а не оценка числа обусловленности; плохо масштабированная обратимая матрица тоже может быть отклонена.

Многопоточный алгоритм и синхронизация

Ключ –threads P задаёт верхнюю границу *общего* числа потоков процесса вместе с основным. Фактическое число участников $p = \min(P, n)$. Создаются $p - 1$ рабочих, основной поток имеет номер 0 и тоже обрабатывает строки. При $P = 1$ дополнительные потоки не создаются. Набор рабочих постоянный на протяжении решения. Это исключает создание и завершение потоков на каждом шаге.

После выбора опоры остаётся $r = n - k - 1$ строк. Участник с номером t получает полуоткрытый интервал

$$\left[k + 1 + \left\lfloor \frac{rt}{p} \right\rfloor, k + 1 + \left\lfloor \frac{r(t+1)}{p} \right\rfloor\right).$$

Интервалы не пересекаются и вместе покрывают все нижние строки. Число строк у участников отличается максимум на одну. В конце алгоритма некоторые интервалы пусты, но все рабочие подтверждают завершение шага.

Состояние набора потоков хранится в структуре Pool: текущий шаг k, номер задания generation, счётчик done и флаг stop. Эти поля защищены одним mutex. Две условные переменные отвечают за новое задание (work) и завершение шага (finished).

Последовательность действий на шаге:

1. Main выбирает и переставляет ведущую строку после завершения прошлого шага.
2. Под mutex записывает k, обнуляет done, увеличивает generation и будит рабочих.
3. Рабочие запоминают k под mutex, отпускают замок и обрабатывают свои строки. Main одновременно вычисляет свой диапазон.
4. Рабочие под mutex увеличивают done. Последний будит main.
5. Main ждёт done=$p - 1$, после чего может менять следующую ведущую строку.

Все ожидания используют while: условие проверяется повторно после пробуждения. Номер задания не даёт повторно выполнить старую работу. Активного ожидания нет. Матрица во время исключения не блокируется: каждый поток пишет только свои строки, ведущая строка доступна всем для чтения. Mutex также обеспечивает видимость записей при передаче задания и завершения. Поэтому нет одновременной записи в один элемент или смены опоры до окончания шага.

Нельзя одновременно выполнять все шаги k: строка следующей опоры зависит от предыдущего исключения. Обратный ход также оставлен последовательным. При завершении main устанавливает stop, будит рабочих и вызывает join для каждого. Только затем уничтожаются средства синхронизации и освобождается память.

Описание программы

Вся реализация C находится в main.c; сборка задаётся CMakeLists.txt. Worker содержит указатель на общий Pool и номер участника. Матрица хранится непрерывно: элемент (i, j) имеет индекс $i(n + 1) + j$. Проверяются выделение памяти, чтение, аргументы, результаты создания потоков, опоры и конечность решения. При ошибке pthread_create уже созданные рабочие корректно завершаются. Для остальных ошибок pthread выводится код и процесс завершается. Отказ ресурсов искусственно не провоцировался.

Основные функции: generate создаёт систему; eliminate обрабатывает диапазон; worker_main выполняет цикл рабочего; start_pool/stop_pool управляют набором; gauss выбирает опоры, согласует шаги и делает обратный ход; verify проверяет ответ.

Вспомогательные number, help, seconds, known_x, check_pthread обслуживают ввод, время и ошибки. main связывает этапы.

Использованы функции POSIX Threads:

- pthread_create, pthread_join — запуск и ожидание завершения;
- pthread_mutex_init/lock/unlock/destroy — защита состояния;
- pthread_cond_init/wait/signal/broadcast/destroy — сон и пробуждение.

Это функции библиотечного интерфейса pthread, а не прямые системные вызовы в исходном коде. pthread_barrier_t не используется, поскольку он не подходит для выбранной переносимой реализации macOS. Таймер — clock_gettime(CLOCK_MONOTONIC), пауза — nanosleep, PID — getpid. Сторонних математических библиотек нет.

Сборка и запуск

Из корня проекта:

```
cmake -S . -B build -DCMAKE_BUILD_TYPE=Release \
  -DCMAKE_C_FLAGS_RELEASE='-O2 -DNDEBUG'
cmake --build build
./build/OS_Lab3_Variant10 --size 1000 --threads 4
./build/OS_Lab3_Variant10 --file tests/three.txt --threads 4
./build/OS_Lab3_Variant10 --help
```

В CLion нужно открыть корневую папку как CMake-проект и задать аргументы в конфигурации запуска. Файл читается в формате: сначала N, затем N строк по N коэффициентов и одному свободному члену. Ключи size/file взаимоисключающие. Допустимы $1 \le n \le 10000$, $1 \le P \le 256$. Для $n = 10000$ только две матрицы занимают около 1,6 ГБ, а вычисление требует большого времени. Для $n = 1000$ две матрицы занимают примерно 15,3 МиБ.

Терминальная сборка CMake и тесты подтверждены. Проект открыт в CLion, исходный файл отображается, CMake загрузился. Глобальный профиль IDE использует GCC 16. Для Apple Clang добавлен отдельный preset apple-clang (CMake 3.21+), его сборка через CMake проверена. После повторной сборки кнопкой Build в CLion получено сообщение Build finished. Сборка IDE проверена в исходном профиле Debug с GCC 16.2.0; основные тесты и замеры выполнены Apple Clang. Глобальные настройки IDE не менялись, preset Apple Clang загружен отдельно. Исходный код использует переносимые интерфейсы C11 и POSIX; в Linux CMake дополнительно подключает стандартную libm. Запуск и сборка в Ubuntu в этой сессии не выполнялись: готовой работающей Linux-среды не было. Виртуальная машина не устанавливалась.

Проверка корректности и примеры

Тестовая система из tests/three.txt:

$$\begin{cases} 2x_1 + x_2 - x_3 = 8, \\ -3x_1 - x_2 + 2x_3 = -11, \\ -2x_1 + x_2 + 2x_3 = -3. \end{cases}$$

Найдено $x = (2, 3, -1)$. При разных лимитах потоков значения совпали с учётом погрешности; для маленьких примеров совпали и напечатанные значения double. Реальный вывод программы:

```
size=3
thread_limit=4
actual_threads=3
time_seconds=0.000022000
residual_inf=8.881784197001e-16
relative_residual=3.062684205863e-17
verification=PASS
x[1]=2.0000000000000004
x[2]=2.9999999999999996
x[3]=-0.99999999999999911
```

Сгенерированные матрицы строго диагонально преобладающие. Линейный конгруэнтный генератор имеет фиксированное начальное состояние 10 и арифметику uint32_t. Эталон $x_j = (j \bmod 7) - 3$, а b вычисляется как Ax_{ref}. Для одного n во всех запусках получаются одинаковые исходные данные.

Проверяется абсолютная невязка $\|Ax - b\|_\infty$ и относительная величина

$$\eta = \frac{\|Ax - b\|_\infty}{\|A\|_\infty \|x\|_\infty + \|b\|_\infty} \le 10^{-10}.$$

Здесь $\|A\|_\infty$ — максимальная сумма модулей элементов строки. Для нулевого знаменателя проверяется сама невязка, переполнение даёт отказ. Для генератора дополнительно требуется $\max_j |x_j - x_{ref,j}| \le 10^{-8}$. Малая невязка не гарантирует малую ошибку x у плохо обусловленной системы. На Apple arm64 тип long double не даёт гарантированного преимущества точности по сравнению с double; проверка на это преимущество не опирается.

Группа проверок	Число
Известные ответы: 2, 3 и 1 неизвестная, pivot, масштаб 10^{-100}, нулевая правая часть; лимиты 1/2/4/8	24
Ошибочные и вырожденные файлы; лимиты 1/4	18
Генерация: n=1,2,3,7,10,31,100,300,600,1000; 1/2/4/8	40
Некорректные аргументы и справка	17
Повторные многопоточные решения n=31, p=8	25
Всего на одну сборку	124

Все 124 проверки пройдены в Release, под AddressSanitizer вместе с UndefinedBehaviorSanitizer и отдельно под ThreadSanitizer. Диагностик санитайзеров не получено. Три теста CTest также прошли. Журналы сохранены в report/results/*-tests.json. Проверка отсутствия гонок дополнена анализом непересекающихся диапазонов и синхронизации шагов; результат TSan не является доказательством для всех входов. Ошибки malloc и pthread_create проверены в коде, но не вызывались искусственным исчерпанием памяти или лимита потоков.

Демонстрация числа потоков

Для наблюдения используется отдельная пауза после создания рабочих и до таймера:

```
./build/OS_Lab3_Variant10 --size 1000 --threads 4 --demo 20 &
pid=$!
ps -M -p "$pid"
wait "$pid"
```

В macOS команда проверена на реальных процессах. После заголовка ps показывает по строке на поток. Во время демопаузы потоки спят, а затем тот же набор потоков выполняет решение. Пауза не входит в измеряемое время.

n	Лимит P	Наблюдалось потоков	Проверка
100	1	1	пройдена
100	2	2	пройдена
100	4	4	пройдена
100	8	8	пройдена
3	8	3	пройдена

Ниже приведён настоящий вывод для P=4. Из длинной первой строки убран только путь COMMAND для читаемости; числовые поля сохранены, полный журнал лежит в threads-p4-n100.txt. Это текстовый снимок, а не изображение терминала.

```
$ ps -M -p 13492
USER PID TT %CPU STAT PRI STIME UTIME
matveyyuri 13492 ?? 0.0 S 31T 0:00.00 0:00.00
           13492 0.0 S 31T 0:00.00 0:00.00
           13492 0.0 S 31T 0:00.00 0:00.00
           13492 0.0 S 31T 0:00.00 0:00.00
```

Отдельный скриншот демонстрации терминала не сохранён. Вместо изображения сохранены исходные логи ps для всех пяти случаев и итоговая таблица JSON. Для Ubuntu подготовлена команда ps -T -p "$pid"; на Ubuntu она в этой сессии не проверялась. Вывод strace не приводится: на macOS он не запускался.

Исследование производительности

Эксперимент выполнен 19.10.2026 непосредственно на MacBook с Apple M1, 8 ГиБ оперативной памяти; macOS 26.5.1 (25F80), arm64. По sysctl доступны 8 физических и 8 логических ядер. Компилятор — Apple Clang 21.0.0 (clang-2100.0.123.102), сборка CMake Release с -O2 -DNDEBUG. Санитайзеры в измерениях отключены. Реальная конфигурация и SHA-256 исходника сохранены в environment.json.

Исследованы n=100,300,600,1000 и p=1,2,4,8. Здесь p включает main и совпадает с фактическим числом участников, поскольку во всех опытах n больше p. Основной поток выполняет вычисления. В конце прямого хода некоторые участники получают пустой диапазон; p означает выделенное число участников, а не гарантированную постоянную занятость каждого.

Для каждой пары выполнен один прогрев и семь учитываемых запусков. Конфигурации перемешаны с фиксированным seed. Одновременно выполнялся один измеряемый процесс; санитайзеры и сборки к этому моменту завершились. Обычные приложения macOS оставались открыты, привязка к ядрам и частоты не управлялись. Поэтому результаты отражают эту рабочую среду и имеют разброс.

Монотонный таймер охватывает всю функцию gauss: выбор опоры, перестановки, исключение строк, синхронизацию и обратный ход. Ввод, генерация, копирование матрицы, создание/завершение набора потоков, проверка, печать и демопауза исключены. Методика одинакова при любом p. Это время решения с готовым набором потоков, а не полное время процесса. Итоговое время — медиана семи наблюдений. Всего сохранено 112 учитываемых измерений и выполнено 16 прогревов.

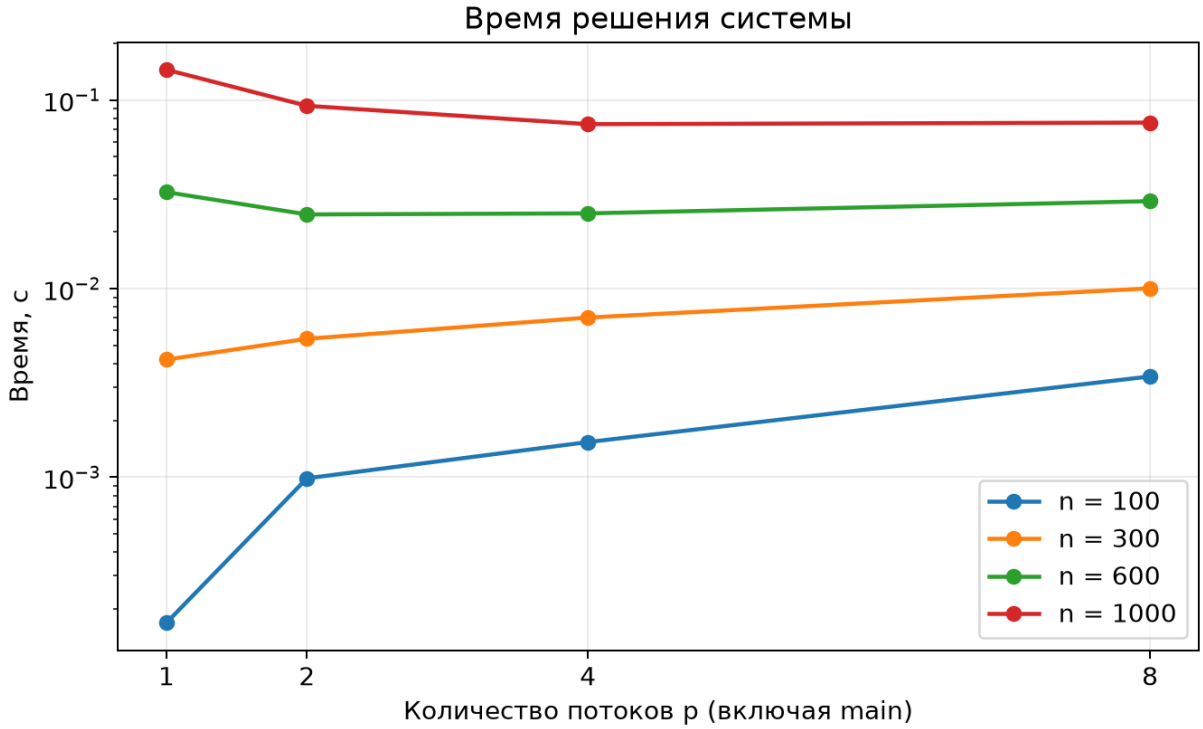
Ускорение и эффективность рассчитаны для каждого размера отдельно:

$$S(p) = \frac{T(1)}{T(p)}, \qquad E(p) = \frac{S(p)}{p}.$$

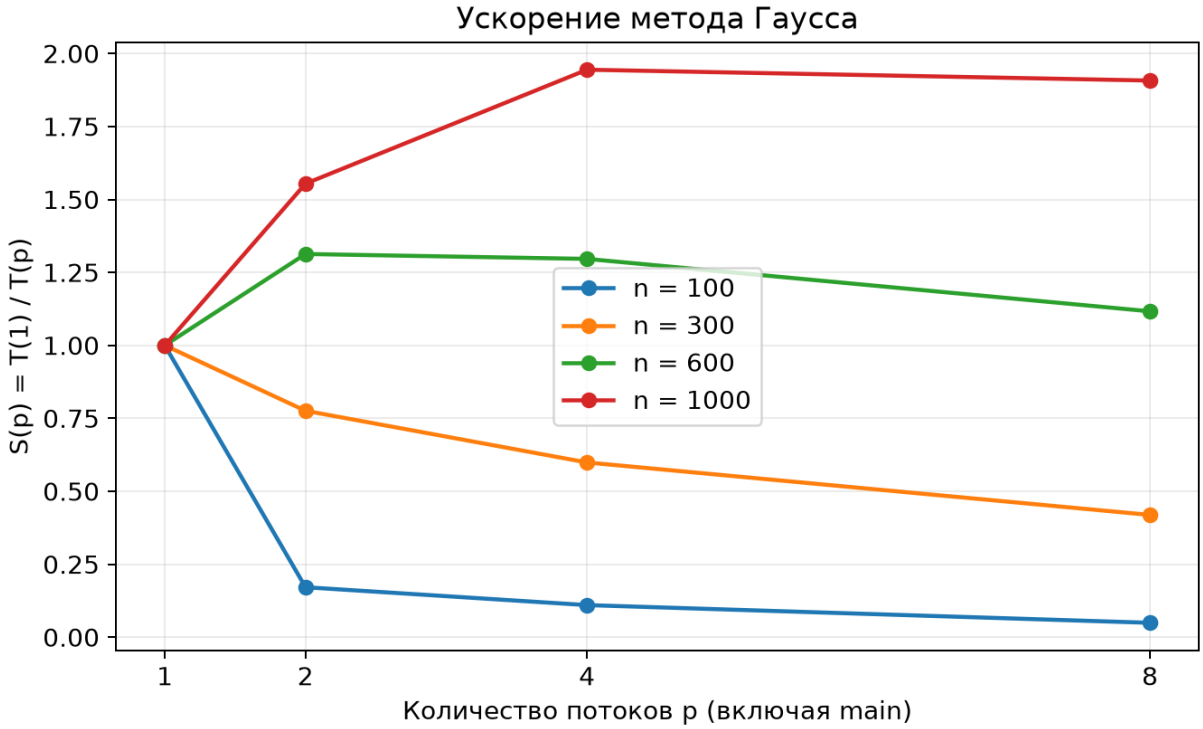
n	p	T, мс	min–max, мс	$S(p)$	$E(p)$, %
100	1	0.169	0.149–0.182	1.000	100.00
100	2	0.987	0.830–1.035	0.171	8.56
100	4	1.536	1.255–1.879	0.110	2.75
100	8	3.423	2.662–3.927	0.049	0.62
300	1	4.206	3.673–4.852	1.000	100.00
300	2	5.421	4.800–7.034	0.776	38.79
300	4	7.024	5.495–7.494	0.599	14.97
300	8	10.037	9.444–10.905	0.419	5.24
600	1	32.494	28.901–37.692	1.000	100.00
600	2	24.746	21.333–25.072	1.313	65.66
600	4	25.066	19.864–28.772	1.296	32.41
600	8	29.097	25.483–34.498	1.117	13.96
1000	1	144.897	133.525–160.774	1.000	100.00
1000	2	93.231	83.296–96.703	1.554	77.71
1000	4	74.512	65.198–78.775	1.945	48.62
1000	8	75.963	71.031–90.424	1.907	23.84

Все времена в таблице указаны в миллисекундах. Исходные данные: raw.csv; агрегированные значения: summary.csv.

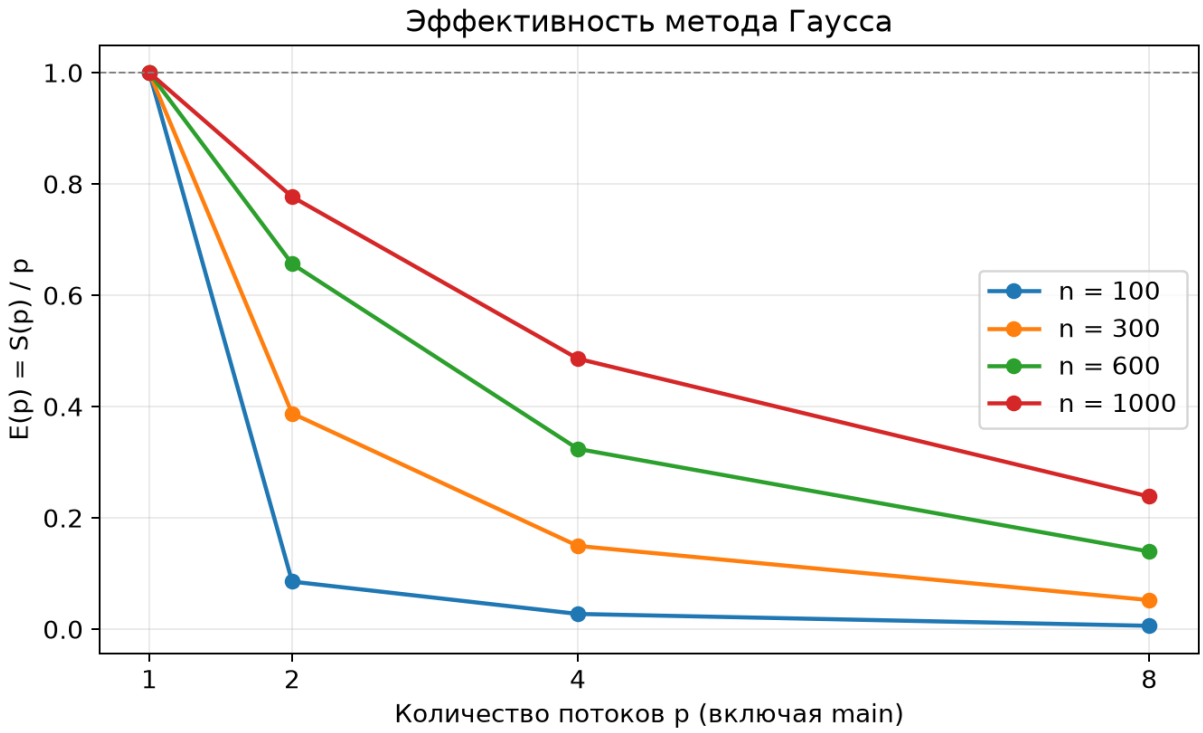
Графики результатов



Время решения от числа потоков. Ось времени логарифмическая, чтобы одновременно были видны все четыре размера.



Ускорение относительно одного потока для каждого n.



Эффективность как доля от идеальной загрузки p участников.

Анализ результатов

При n=100 последовательное решение занимает около 0,169 мс. Для четырёх потоков время 1,536 мс, ускорение 0,110: решение медленнее примерно в 9,09 раза. Работы в отдельных строках мало, а ожидание, пробуждение и блокировки требуются почти на каждом шаге. Постоянный набор устраняет повторные create/join, но не устраняет эти расходы. При n=300 многопоточность тоже медленнее.

При n=600 два потока дают ускорение около 1,313, четыре — 1,296. При n=1000 время уменьшается с 0,144897 с до 0,074512 с на четырёх потоках: $S(4) \approx 1,945$, $E(4) \approx 0,486$, то есть около 48,6%. Восемь потоков дают $S(8) \approx 1,907$ и $E(8) \approx 0,238$. Разница медиан для четырёх и восьми потоков невелика, а интервалы минимума–максимума пересекаются; строгий вывод о превосходстве четырёх потоков при любой фоновой нагрузке делать нельзя.

У больших матриц объём вычислений растёт как $O(n^3)$, и расходы ожидания лучше окупаются. Линейного ускорения нет: выбор опоры и обратный ход последовательны, каждый шаг требует согласования, число нижних строк уменьшается, а память и вычислительные ресурсы общие. Размещение потоков планировщиком и фоновые задачи также могут влиять на время. Частоты, нагрузка по конкретным ядрам и вклад каждого фактора отдельно не измерялись. Наличие восьми физических ядер само по себе не обещает ускорения в восемь раз.

Выводы

Реализован метод Гаусса с частичным выбором ведущего элемента на C11 и переносимой синхронизацией pthread. Основной и рабочие потоки действительно обрабатывают разные строки. Лимит потоков соблюдается и подтверждён ps. Проверены известные решения, перестановки, вырожденные и ошибочные системы; ASan/UBSan и TSan прошли выполненные тесты без диагностик.

Многопоточность полезна при достаточном объёме работы. В этой серии опытов при n=1000 четыре потока дали ускорение около 1,94, а маленькие матрицы выгоднее решать одним потоком. Измерения, графики и исходный код сохранены для повторения. Неподтверждённые проверки: запуск и команда ps на Ubuntu, искусственные отказы ресурсов. Скриншот терминала заменён настоящими текстовыми журналами. Личные данные титульного листа необходимо заполнить перед сдачей.

Источники

1. Предоставленное задание «Лабораторная работа 3.pdf», вариант 10.
2. Шаблон преподавателя: https://github.com/Yadroff/OS_report_template. Просмотрен 19.10.2026.
3. Исходный код main.c и журналы экспериментов report/results данного проекта.

Исходная программа

Полный код C11 из main.c. Номера строк соответствуют исходному файлу.

```c
#define _POSIX_C_SOURCE 200809L
#include <errno.h>
#include <float.h>
#include <math.h>
#include <pthread.h>
#include <stdint.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <time.h>
#include <unistd.h>

typedef struct {
    double *a;
    int n, threads, k, generation, done, stop;
    pthread_mutex_t mutex;
    pthread_cond_t work, finished;
} Pool;

typedef struct {
    Pool *pool;
    int id;
} Worker;

static void check_pthread(int error, const char *name) {
    if (error != 0) {
        fprintf(stderr, "%s: %s\n", name, strerror(error));
        exit(EXIT_FAILURE);
    }
}

static void help(void) {
    puts("Usage: OS_Lab3_Variant10 (--size N | --file PATH) --threads P [--demo SEC]\n"
         " --size N Generate a reproducible system, 1 <= N <= 10000\n"
         " --file PATH Read N, then N rows of N coefficients and a right side\n"
         " --threads P Total thread limit including main, 1 <= P <= 256\n"
         " --demo SEC Pause with workers alive before timing, 1..300 seconds\n"
         " --help Show this help\n"
         "Actual threads = min(P, N). Times exclude input, pool setup and checking.");
}

static int number(const char *s, int limit) {
    char *end;
    errno = 0;
    long value = strtol(s, &end, 10);
    if (errno || end == s || *end || value < 1 || value > limit) return 0;
    return (int)value;
}

static double seconds(void) {
    struct timespec t;
    if (clock_gettime(CLOCK_MONOTONIC, &t) != 0) {
        perror("clock_gettime");
        exit(EXIT_FAILURE);
    }
    return (double)t.tv_sec + (double)t.tv_nsec / 1e9;
}

static double known_x(int j) {
    return (double)(j % 7 - 3);
}

static void generate(double *a, int n) {
    uint32_t seed = 10;
    size_t width = (size_t)n + 1;
    for (int i = 0; i < n; ++i) {
        double sum = 0, b = 0;
        for (int j = 0; j < n; ++j) {
            seed = seed * UINT32_C(1664525) + UINT32_C(1013904223);
            double value = (double)((int)(seed % 21) - 10);
```

```
            a[(size_t)i * width + j] = value;
            if (i != j) sum += fabs(value);
        }
        a[(size_t)i * width + i] = sum + 1;
        for (int j = 0; j < n; ++j)
            b += a[(size_t)i * width + j] * known_x(j);
        a[(size_t)i * width + n] = b;
    }
}

/* Each participant changes a disjoint, contiguous range below row k. */
static void eliminate(Pool *pool, int id, int k) {
    int n = pool->n;
    size_t width = (size_t)n + 1;
    int rows = n - k - 1;
    int start = k + 1 + rows * id / pool->threads;
    int end = k + 1 + rows * (id + 1) / pool->threads;
    const double *pivot = pool->a + (size_t)k * width;
    for (int i = start; i < end; ++i) {
        double *row = pool->a + (size_t)i * width;
        double factor = row[k] / pivot[k];
        row[k] = 0;
        for (int j = k + 1; j <= n; ++j)
            row[j] -= factor * pivot[j];
    }
}

static void *worker_main(void *arg) {
    Worker *worker = arg;
    Pool *pool = worker->pool;
    int seen = 0;
    check_pthread(pthread_mutex_lock(&pool->mutex), "mutex_lock");
    for (;;) {
        while (!pool->stop && pool->generation == seen)
            check_pthread(pthread_cond_wait(&pool->work, &pool->mutex), "cond_wait");
        if (pool->stop) break;
        seen = pool->generation;
        int k = pool->k;
        check_pthread(pthread_mutex_unlock(&pool->mutex), "mutex_unlock");
        eliminate(pool, worker->id, k);
        check_pthread(pthread_mutex_lock(&pool->mutex), "mutex_lock");
        ++pool->done;
        if (pool->done == pool->threads - 1)
            check_pthread(pthread_cond_signal(&pool->finished), "cond_signal");
    }
    check_pthread(pthread_mutex_unlock(&pool->mutex), "mutex_unlock");
    return NULL;
}

static void stop_pool(Pool *pool, pthread_t *ids, int created) {
    check_pthread(pthread_mutex_lock(&pool->mutex), "mutex_lock");
    pool->stop = 1;
    check_pthread(pthread_cond_broadcast(&pool->work), "cond_broadcast");
    check_pthread(pthread_mutex_unlock(&pool->mutex), "mutex_unlock");
    for (int i = 0; i < created; ++i)
        check_pthread(pthread_join(ids[i], NULL), "pthread_join");
    check_pthread(pthread_cond_destroy(&pool->finished), "cond_destroy");
    check_pthread(pthread_cond_destroy(&pool->work), "cond_destroy");
    check_pthread(pthread_mutex_destroy(&pool->mutex), "mutex_destroy");
}

static int start_pool(Pool *pool, pthread_t *ids, Worker *workers) {
    check_pthread(pthread_mutex_init(&pool->mutex, NULL), "mutex_init");
    check_pthread(pthread_cond_init(&pool->work, NULL), "cond_init");
    check_pthread(pthread_cond_init(&pool->finished, NULL), "cond_init");
    for (int i = 0; i < pool->threads - 1; ++i) {
        workers[i].pool = pool;
        workers[i].id = i + 1;
        int error = pthread_create(&ids[i], NULL, worker_main, &workers[i]);
        if (error != 0) {
            fprintf(stderr, "pthread_create: %s\n", strerror(error));
            stop_pool(pool, ids, i);
            return 0;
```

```
144             }
145         }
146         return 1;
147     }
148
149     static int gauss(Pool *pool, double *x) {
150         int n = pool->n;
151         double *a = pool->a;
152         size_t width = (size_t)n + 1;
153         double scale = 0;
154         for (int i = 0; i < n; ++i)
155             for (int j = 0; j < n; ++j)
156                 scale = fmax(scale, fabs(a[(size_t)i * width + j]));
157         double tolerance = DBL_EPSILON * n * scale;
158         for (int k = 0; k < n; ++k) {
159             int best = k;
160             for (int i = k + 1; i < n; ++i)
161                 if (fabs(a[(size_t)i * width + k]) > fabs(a[(size_t)best * width + k]))
162                     best = i;
163             double pivot = fabs(a[(size_t)best * width + k]);
164             if (!isfinite(pivot) || pivot <= tolerance) return 0;
165             if (best != k) {
166                 for (int j = 0; j <= n; ++j) {
167                     double tmp = a[(size_t)k * width + j];
168                     a[(size_t)k * width + j] = a[(size_t)best * width + j];
169                     a[(size_t)best * width + j] = tmp;
170                 }
171             }
172             if (k == n - 1) break;
173             if (pool->threads == 1) {
174                 eliminate(pool, 0, k);
175             } else {
176                 check_pthread(pthread_mutex_lock(&pool->mutex), "mutex_lock");
177                 pool->k = k;
178                 pool->done = 0;
179                 ++pool->generation;
180                 check_pthread(pthread_cond_broadcast(&pool->work), "cond_broadcast");
181                 check_pthread(pthread_mutex_unlock(&pool->mutex), "mutex_unlock");
182                 eliminate(pool, 0, k);
183                 check_pthread(pthread_mutex_lock(&pool->mutex), "mutex_lock");
184                 while (pool->done != pool->threads - 1)
185                     check_pthread(pthread_cond_wait(&pool->finished, &pool->mutex), "cond_wait");
186                 check_pthread(pthread_mutex_unlock(&pool->mutex), "mutex_unlock");
187             }
188         }
189         for (int i = n - 1; i >= 0; --i) {
190             double sum = a[(size_t)i * width + n];
191             for (int j = i + 1; j < n; ++j) sum -= a[(size_t)i * width + j] * x[j];
192             x[i] = sum / a[(size_t)i * width + i];
193             if (!isfinite(x[i])) return 0;
194         }
195         return 1;
196     }
197
198     static int verify(const double *a, const double *x, int n, int generated) {
199         long double residual = 0, norm_a = 0, norm_b = 0, norm_x = 0, error = 0;
200         size_t width = (size_t)n + 1;
201         for (int i = 0; i < n; ++i) {
202             long double sum = 0, row_sum = 0;
203             for (int j = 0; j < n; ++j) {
204                 sum += (long double)a[(size_t)i * width + j] * x[j];
205                 row_sum += fabsl(a[(size_t)i * width + j]);
206             }
207             if (!isfinite(sum) || !isfinite(row_sum)) {
208                 puts("verification=FAIL (numeric overflow during checking)");
209                 return 0;
210             }
211             residual = fmaxl(residual, fabsl(sum - a[(size_t)i * width + n]));
212             norm_a = fmaxl(norm_a, row_sum);
213             norm_b = fmaxl(norm_b, fabsl(a[(size_t)i * width + n]));
214             norm_x = fmaxl(norm_x, fabsl(x[i]));
215             if (generated) error = fmaxl(error, fabsl(x[i] - known_x(i)));
216         }
```

```
    long double denominator = norm_a * norm_x + norm_b;
    long double relative = denominator > 0 ? residual / denominator : residual;
    int ok = isfinite(denominator) && isfinite(relative) && relative <= 1e-10L;
    if (generated && error > 1e-8L) ok = 0;
    printf("residual_inf=%.12Le\nrelative_residual=%.12Le\n", residual, relative);
    if (generated) printf("max_solution_error=%.12Le\n", error);
    printf("verification=%s\n", ok ? "PASS" : "FAIL");
    return ok;
}

int main(int argc, char **argv) {
    int n = 0, threads = 0, demo = 0;
    const char *file = NULL;
    for (int i = 1; i < argc; ++i) {
        if (strcmp(argv[i], "--help") == 0 && argc == 2) { help(); return 0; }
        if (i + 1 == argc) goto bad_args;
        const char *option = argv[i++];
        if (strcmp(option, "--size") == 0 && !n && !file) {
            n = number(argv[i], 10000);
            if (!n) goto bad_args;
        } else if (strcmp(option, "--file") == 0 && !file && !n) {
            file = argv[i];
        } else if (strcmp(option, "--threads") == 0 && !threads) {
            threads = number(argv[i], 256);
            if (!threads) goto bad_args;
        } else if (strcmp(option, "--demo") == 0 && !demo) {
            demo = number(argv[i], 300);
            if (!demo) goto bad_args;
        } else goto bad_args;
    }
    if ((!n && !file) || !threads) goto bad_args;
    FILE *input = NULL;
    if (file) {
        input = fopen(file, "r");
        if (!input) { perror(file); return 1; }
        char token[128];
        if (fscanf(input, "%127s", token) != 1 || !(n = number(token, 10000))) {
            fprintf(stderr, "Invalid matrix size in file.\n");
            fclose(input);
            return 1;
        }
    }
    size_t count = (size_t)n * ((size_t)n + 1);
    double *a = malloc(count * sizeof(*a));
    double *original = malloc(count * sizeof(*original));
    double *x = malloc((size_t)n * sizeof(*x));
    int actual = threads < n ? threads : n;
    pthread_t *ids = calloc((size_t)actual, sizeof(*ids));
    Worker *workers = calloc((size_t)actual, sizeof(*workers));
    int result = 1;
    if (!a || !original || !x || !ids || !workers) {
        fprintf(stderr, "Not enough memory.\n");
        if (input) fclose(input);
        goto cleanup;
    }
    if (input) {
        for (size_t i = 0; i < count; ++i) {
            if (fscanf(input, "%lf", &a[i]) != 1 || !isfinite(a[i])) {
                fprintf(stderr, "Expected finite matrix coefficients and right sides.\n");
                fclose(input);
                goto cleanup;
            }
        }
        char extra[2];
        int extra_result = fscanf(input, "%1s", extra);
        int read_error = ferror(input);
        fclose(input);
        if (extra_result == 1 || read_error) {
            fprintf(stderr, "Extra data or read error in matrix file.\n");
            goto cleanup;
        }
    } else generate(a, n);
    memcpy(original, a, count * sizeof(*a));
```

```
    Pool pool = {.a = a, .n = n, .threads = actual};
    if (!start_pool(&pool, ids, workers)) goto cleanup;
    printf("size=%d\nthread_limit=%d\nactual_threads=%d\n", n, threads, actual);
    if (demo) {
        printf("demo_pid=%ld\ndemo_seconds=%d\n", (long)getpid(), demo);
        fflush(stdout);
        struct timespec delay = {.tv_sec = demo, .tv_nsec = 0};
        while (nanosleep(&delay, &delay) == -1 && errno == EINTR) {}
    }
    double start = seconds();
    int solved = gauss(&pool, x);
    double elapsed = seconds() - start;
    stop_pool(&pool, ids, actual - 1);
    printf("time_seconds=%.9f\n", elapsed);
    if (!solved) {
        fprintf(stderr, "Singular or numerically unsafe system: unique solution unavailable.\n");
        result = 2;
        goto cleanup;
    }
    result = verify(original, x, n, file == NULL) ? 0 : 3;
    if (n <= 10)
        for (int i = 0; i < n; ++i) printf("x[%d]=%.17g\n", i + 1, x[i]);
cleanup:
    free(workers);
    free(ids);
    free(x);
    free(original);
    free(a);
    return result;
bad_args:
    fprintf(stderr, "Invalid arguments. Use --help.\n");
    return 1;
}
```